UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA PRÁCTICA SEMANA 05

Asignatura: Desarrollo de Aplicaciones Web (IS093A)
Unidad I: Desarrollo Web Frontend
Tema: Framework JS (React), Client-Side Rendering (CSR), JSX, Props & Children, Estrategias de Estilado
Duración: 90 minutos

APELLIDOS Y NOMBRES: _____

OBJETIVO DE LA PRÁCTICA

Crear una aplicación frontend de página única (SPA) utilizando un framework JS (React), configurar el entorno con herramientas modernas (Vite + npm/pnpm), implementar arquitectura basada en componentes con JSX, transmitir datos mediante props y composición con children, aplicar estrategias de estilado (CSS Modules / Tailwind / inline) y validar el comportamiento de **Client-Side Rendering (CSR)** frente al ciclo de vida del DOM.

RECURSOS REQUERIDOS

- Node.js LTS (≥20.x) + npm/pnpm
- Visual Studio Code (Extensiones: ES7+ React/Redux snippets, Tailwind CSS IntelliSense, Error Lens, React Developer Tools)
- Navegador moderno + DevTools + extensión React DevTools
- Terminal integrada
- Documentación oficial: react.dev, vitejs.dev
- Conexión a internet

PARTE 1: EJERCICIO PASO A PASO

Paso	Actividad	Tiempo	Concepto Clave	Restricción/Nota Técnica
1	Scaffold del proyecto: npm create vite@latest, seleccionar React + JavaScript, instalar dependencias,	7 min	Gestión de dependencias, bundler (Vite), HMR, estructura de carpetas	No usar create-react-app (obsoleto). Verificar que package.json tenga

	ejecutar npm run dev			@vitejs/plugin-react
2	Limpiar boilerplate, crear src/components/, src/styles/, configurar alias o rutas relativas. Estructurar App.jsx como orquestador	8 min	Arquitectura de componentes, árbol de render, JSX → React.createElement	Mantener un solo componente raíz <App> que importa hijos. Evitar lógica de negocio en App
3	Implementar patrón props y children: crear <Dashboard> y <MetricCard>. Pasar datos estructurados y contenido anidado	12 min	Flujo unidireccional, inmutabilidad de props, composición vs herencia, PropTypes/JSDoc	Documentar tipos con JSDoc /** @param {Object} props */. No usar state en este paso
4	Aplicar estrategias de estilado: CSS Modules para <MetricCard>, utilidades Tailwind (si se configura) o inline para contrastar scope y colisiones	10 min	Scope de estilos, className vs class, CSS Modules hashing, prioridad de cascada en CSR	Prohibido usar <style> global en componentes. Usar *.module.css explícito
5	Validar CSR: inspeccionar HTML inicial (View Page Source), verificar hidratación en React DevTools, analizar red (JS	8 min	CSR vs SSR, hidratación, bundle analysis, index.html vacío vs DOM inyectado	Explicar por qué meta viewport y SEO tags deben ir en index.html, no en JSX

	chunks) y ciclo de render			

USO DE HERRAMIENTAS DE IA (Recomendado para optimizar el ejercicio)

Herramienta	Uso Permitido	Prompt Ejemplo	Uso Prohibido
ChatGPT/Claude /Copilot	Explicar diferencias props vs children, depurar errores de hidratación, generar vite.config.js base	"¿Por qué mi componente hijo no recibechildrensi usoprops.childrenexplícitamente?"	"Genera la app completa de dashboard con React y Tailwind"
React DevTools	Inspeccionar árbol de componentes, verificar props pasadas, medir re-renders	(Extensión oficial)	Modificar estado directamente desde DevTools para "arreglar" la UI
Bundle Analyzer / Lighthouse	Medir tamaño de chunks JS, impacto de CSR en FCP/LCP	(Automático)	Ignorar advertencias de Missing key prop in list o Invalid prop type

Regla de laboratorio: El 75% de la estructura de componentes y transmisión de datos debe escribirse manualmente. Cada uso de IA para resolver conflictos de estilos o flujo de props debe llevar comentario // IA: [problema] → Solución manual: [explicación]

Evidencias del Ejercicio Desarrollado:

- Añada capturas de pantalla de cada paso realizado en Visual Studio Code.
- Añada capturas de la Página Web implementada de manera local
- Proporcione el enlace de Github de su proyecto